

UNIT-IV

INTERMEDIATE SQL

1. FOREIGN KEY

What is Foreign Key?

- A Foreign Key is a column used to connect two tables.
- It refers to the Primary Key of another table.
- It helps maintain data integrity and security.

Example:

- Users table → id
- Orders table → user_id
- Orders.user_id refers to Users.id.

Uses

- Connects two tables.
- Prevents invalid data.
- Protects related records from accidental deletion/update.

Basic Syntax

```
ALTER TABLE Orders  
ADD CONSTRAINT orders_user_id_key  
FOREIGN KEY (user_id)  
REFERENCES Users(id);
```

2. Protection from DELETE and UPDATE

DELETE

- If a user has related orders, deleting that user can cause data problems.

NO ACTION / RESTRICT:

- Prevents deletion if related records exist.

UPDATE

- If the user's ID is changed, related order records may become invalid.
- So, the Foreign Key constraint prevents the update.

3. Referential Actions

Main types:

Action	Meaning
NO ACTION	Prevents invalid DELETE/UPDATE
RESTRICT	Restricts DELETE/UPDATE
CASCADE	Automatically deletes related records
SET NUL	SET NULL Sets related foreign key to NULL

Example

FOREIGN KEY (user_id)

REFERENCES Users(id)

ON DELETE CASCADE

ON UPDATE CASCADE;

4.TRANSACTIONS

What is Transaction?

A Transaction is a group of SQL statements treated as one unit.

It is useful when:

- All operations must succeed together.
- If an error occurs, all changes should be cancelled.

Important Commands

START TRANSACTION

START TRANSACTION;

Starts a transaction.

COMMIT

COMMIT;

Permanently saves the changes.

ROLLBACK

ROLLBACK;

Cancels the changes and returns to the previous state.

Example

START TRANSACTION;

UPDATE Products

SET price = 500

WHERE id = 5;

UPDATE Products

SET price = 500

WHERE id = 6;

COMMIT;

If an error occurs:

ROLLBACK;

5.ACID

- ACID represents four important properties of transactions.

Letter	Full Form	Meaning
A	Atomicity	All operations succeed or all fail
C	Consistency	Data remains correct
I	Isolation	Transactions do not interfere with each other
D	Durability	Committed data is permanently saved

Social Media Analytics – Complete SQL Script

1.Database Design Overview

Entities

- Users
- Platforms (Instagram, X, LinkedIn, etc.)
- Posts
- Reactions (Likes, Comments, Shares)
- Followers

2.Table Creation Script (DDL)

Users Table

```
CREATE TABLE users (user_id INT PRIMARY KEY,username VARCHAR(50) NOT NULL,join_date DATE,country VARCHAR(50));
```

Platforms Table

```
CREATE TABLE platforms ( platform_id INT PRIMARY KEY, platform_name VARCHAR(30) UNIQUE NOT NULL );
```

Posts Table

```
CREATE TABLE posts ( post_id INT PRIMARY KEY, user_id INT, platform_id INT, post_date DATE, content_type VARCHAR(20), -- image, video, reel, text CONSTRAINT fk_post_user FOREIGN KEY (user_id) REFERENCES users(user_id), CONSTRAINT fk_post_platform FOREIGN KEY (platform_id) REFERENCES platforms(platform_id) );
```

Reactions Table

```
CREATE TABLE reactions ( reaction_id INT PRIMARY KEY, post_id INT, reaction_type VARCHAR(20), -- like, comment, share reaction_date DATE, CONSTRAINT fk_reaction_post FOREIGN KEY (post_id) REFERENCES posts(post_id) );
```

Followers Table

```
CREATE TABLE followers ( follower_id INT, following_id INT, follow_date DATE, PRIMARY
KEY (follower_id, following_id), CONSTRAINT fk_follower FOREIGN KEY (follower_id)
REFERENCES users(user_id), CONSTRAINT fk_following FOREIGN KEY (following_id)
REFERENCES users(user_id) );
```

3.Sample Data Insertion (DML)

Users

```
INSERT INTO users VALUES (1, 'arjun_ai', DATE '2024-01-10', 'India');
INSERT INTO users VALUES (2, 'meera_data', DATE '2024-02-15', 'India');
INSERT INTO users VALUES (3, 'john_ml', DATE '2023-11-05', 'USA');
INSERT INTO users VALUES (4, 'sara_cloud', DATE '2024-03-01', 'UK');
```

Platforms

```
INSERT INTO platforms VALUES (1, 'Instagram');
INSERT INTO platforms VALUES (2, 'X');
INSERT INTO platforms VALUES (3, 'LinkedIn');
```

Posts

```
INSERT INTO posts VALUES (101, 1, 1, DATE '2024-06-01', 'reel');
INSERT INTO posts VALUES (102, 2, 1, DATE '2024-06-02', 'image');
INSERT INTO posts VALUES (103, 3, 2, DATE '2024-06-03', 'text');
INSERT INTO posts VALUES (104, 1, 3, DATE '2024-06-04', 'article');
```

Reactions

```
INSERT INTO reactions VALUES (1001, 101, 'like', DATE '2024-06-01');
INSERT INTO reactions VALUES (1002, 101, 'comment', DATE '2024-06-01');
INSERT INTO reactions VALUES (1003, 101, 'share', DATE '2024-06-02');
INSERT INTO reactions VALUES (1004, 102, 'like', DATE '2024-06-02');
INSERT INTO reactions VALUES (1005, 103, 'like', DATE '2024-06-03');
INSERT INTO reactions VALUES (1006, 104, 'comment', DATE '2024-06-04');
```

Followers

```
INSERT INTO followers VALUES (2, 1, DATE '2024-05-01');
```

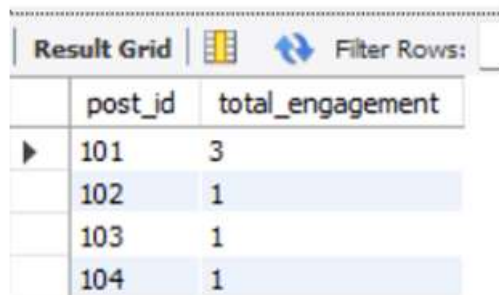
```
INSERT INTO followers VALUES (3, 1, DATE '2024-05-02');
```

```
INSERT INTO followers VALUES (4, 2, DATE '2024-05-03');
```

Analytics Queries (Core Learning Part)

1. Total Engagement per Post

```
SELECT p.post_id, COUNT(r.reaction_id) AS total_engagement FROM posts p LEFT JOIN  
reactions r ON p.post_id = r.post_id GROUP BY p.post_id;
```



The screenshot shows a database interface with a 'Result Grid' tab. It displays the results of a SQL query. The grid has two columns: 'post_id' and 'total_engagement'. There are four rows of data. The first row shows post_id 101 with a total_engagement of 3. The second row shows post_id 102 with a total_engagement of 1. The third row shows post_id 103 with a total_engagement of 1. The fourth row shows post_id 104 with a total_engagement of 1. The interface includes a 'Filter Rows' button and a small icon of a document with a list.

	post_id	total_engagement
▶	101	3
	102	1
	103	1
	104	1

2. Engagement Breakdown (Like / Comment / Share)

```
SELECT p.post_id, r.reaction_type, COUNT(*) AS reaction_count FROM posts p JOIN  
reactions r ON p.post_id = r.post_id GROUP BY p.post_id, r.reaction_type;
```

Result Grid			
Filter Rows:			
	post_id	reaction_type	reaction_count
▶	101	like	1
	101	comment	1
	101	share	1
	104	comment	1
	102	like	1
	103	like	1

3. Top 5 Most Engaging Posts

```
SELECT * FROM ( SELECT p.post_id, COUNT(r.reaction_id) AS engagement, RANK() OVER
(ORDER BY COUNT(r.reaction_id) DESC) AS rnk FROM posts p LEFT JOIN reactions r ON
p.post_id = r.post_id GROUP BY p.post_id ) WHERE rnk <= 5;
```

Result Grid			
Filter Rows:			
	post_id	engagement	rnk
▶	101	3	1
	104	1	2
	102	1	2
	103	1	2

4. Most Engaging Users

```
SELECT u.username, COUNT(r.reaction_id) AS total_engagement FROM users u JOIN
posts p ON u.user_id = p.user_id LEFT JOIN reactions r ON p.post_id = r.post_id GROUP BY
u.username ORDER BY total_engagement DESC;
```

	username	total_engagement
▶	arjun_ai	4
	meera_data	1
	john_ml	1

5. Users Who Never Posted

```
SELECT u.username FROM users u LEFT JOIN posts p ON u.user_id = p.user_id WHERE
p.post_id IS NULL;
```

Result Grid		Filter
	username	
▶	sara_cloud	

6. Average Reactions per Post

```
SELECT AVG(reaction_count) AS avg_reactions FROM ( SELECT post_id, COUNT(*) AS
reaction_count FROM reactions GROUP BY post_id );
```

Result Grid		Filter
	avg_reactions	
▶	1.5000	

7. Platform-wise Engagement

```
SELECT pl.platform_name, COUNT(r.reaction_id) AS total_engagement FROM platforms pl
JOIN posts p ON pl.platform_id = p.platform_id LEFT JOIN reactions r ON p.post_id =
r.post_id GROUP BY pl.platform_name;
```

Result Grid		Filter Rows:
	platform_name	total_engagement
▶	Instagram	4
	LinkedIn	1
	X	1

8. Peak Posting Day

```
SELECT post_date, COUNT(*) AS post_count FROM posts GROUP BY post_date ORDER BY
post_count DESC;
```

Result Grid			Filter Rows:
	post_date	post_count	
▶	2024-06-01	1	
	2024-06-02	1	
	2024-06-03	1	
	2024-06-04	1	

9. Users with More Than 1,000 Engagements (Scalable Query)

```
SELECT u.username FROM users u JOIN posts p ON u.user_id = p.user_id JOIN reactions r
ON p.post_id = r.post_id GROUP BY u.username HAVING COUNT(r.reaction_id) > 1000;
```

Result Grid		Filter Rows:
	username	

10. Followers Count per User

```
SELECT u.username, COUNT(f.follower_id) AS follower_count FROM users u LEFT JOIN
followers f ON u.user_id = f.following_id GROUP BY u.username;
```

Result Grid			Filter Rows:
	username	follower_count	
▶	arjun_ai	2	
	meera_data	1	
	john_ml	0	
	sara_cloud	0	